ChatGPT

Structured Data Overview

The `dump_structured_v2.json` file organizes the APA league data into several top-level sections. Key elements include **meta**, **league**, **divisions**, **teams**, **players**, **matches**, **locations**, and auxiliary fields. For example, the transform script builds the JSON with these keys [1]. Briefly:

- **Meta & League:** Contains global info about the league and session. The `"meta"` object holds things like `league_name`, `session` (season name), creation date and a `league_details` sub-object. The `"league"` object has the league's ID, slug, name, and other fields (phone, homepage, contacts) as returned from APA's API [2]. In fact, the code sets:

```
"league": {
  "id": <league_id>,
  "slug": <league_slug>,
  "name": <league_name>,
  "is_default": <bool>,
  "raw": { … }
}
```

  The meta object also carries `league_details` (id, name, slug, number, phone, URLs, etc.) drawn from the API [2].

- **Divisions:** Under `"divisions"`, each key is a **division slug** (often derived from the division name and format). Each division entry includes fields like `id` (the slug), `apa_division_code` (the official APA division number), `name`, and `format` (e.g. `"8-ball"` or `"9-ball"`) [3]. It also records the session and play night, plus a standings list and team IDs. For example, the code constructs:

```
"divisions": {
  "<div_slug>": {
    "id": "<div_slug>",
    "apa_division_code": <division_number>,
    "name": "<Division Name>",
    "format": "8-ball",
    "session": "Spring 2024",
    "night": "Tuesday",
    "standings": [ ... ],
    "team_ids": [ ... ],
    "urls": { "standings": "<page_url>" },
    "apa_meta": { ... }
  },
```

```
  ...
}
```

(Division slugs often append `_8` or `_9` if there are both 8- and 9-ball divisions with similar names.) The transform code stores each division by slug and fills in APA metadata and team lists [3].

- **Teams:** Under `"teams"`, keys are **team slugs** (unique IDs generated from team names, with `_8` or `_9` suffix to indicate format if needed [4]). Each team object contains its `id` (slug), `name`, `format`, and `division_id` (the slug of its division) [5]. It also includes the home location (name and `home_location_id`), a `roster` array, and some session summary. For example:

```
"teams": {
  "<team_slug>": {
    "id": "<team_slug>",
    "name": "Team Name",
    "format": "8-ball",
    "division_id": "<div_slug>",
    "home_location": "Bar Name",
    "home_location_id": "<loc_id>",
    "roster": [
      { "player_id": "<player_slug>", "alias_id": 1234567 },
      ...
    ],
    "session_summary": { ... },
    "urls": { "team_page": "<page_url>" },
    "apa_meta": { ... }
  },
  ...
}
```

The `roster` is a list of players on the team, each with a `player_id` (the player's slug) and `alias_id` (the APA alias ID) [6] [5]. The `apa_meta` field holds raw APA IDs (team number, division ID, etc.) for reference.

- **Players:** Under `"players"`, each key is a **player slug**. The slug is usually the player's name lowercased (e.g. `"john_smith"`), or if no name is available, `"alias_<id>"`. Each player entry includes:

```
"players": {
  "<player_slug>": {
    "id": "<player_slug>",
    "alias_id": <alias_id or null>,
    "member_id": <member_id or null>,
    "display_name": "First Last",
    "stats": {
```

```
        "8-ball": { ... },
        "9-ball": { ... }
    },
    "profile": {
        "first_name": "First",
        "last_name": "Last",
        "consecutive_years_played": ...,
        "membership_history": [...],
        "raw_alias": { ... }
    }
  },
  ...
}
```

(Fields come from the APA API for alias and player stats.) The `alias_id` is the APA alias identifier, and `member_id` is the member's ID if available. Crucially, each player's `stats` are split by format. For example, the code merges 8-ball stats into `"8-ball"` and 9-ball stats into `"9-ball"` for each player [7]. So one person's 8-ball and 9-ball records live under separate keys in `"stats"`. The `profile` object stores personal info (names, years played, membership history, etc.) from the alias profile API call [8].

- **Matches (Scorecards):** Under `"matches"`, each key is a **match slug** of the form `"match_<date>_<fmt>_<home>_<away>"`. Each match object includes:

```
"matches": {
  "<match_slug>": {
    "id": "<match_slug>",
    "apa_match_id": <APA match ID>,
    "date": "YYYY-MM-DD",
    "division_id": "<div_slug>",
    "format": "8-ball",
    "location": "Location Name",
    "home_team_id": "<home_team_slug>",
    "away_team_id": "<away_team_slug>",
    "team_scores": {
        "home_subtotal": <points>, "away_subtotal": <points>,
        "home_bonus": <points>, "away_bonus": <points>,
        "home_total": <points>, "away_total": <points>
    },
    "sets": [
      {
        "set_order": 1,
        "home_player_id": "<player_slug>",
        "away_player_id": "<player_slug>",
        "home_skill_level": 3,
        "away_skill_level": 4,
```

```
        "home_points": <pts>, "away_points": <pts>,
        "home_win_loss": "W", "away_win_loss": "L",
        "home_eight_ball_match_points_earned": <pts>,
        "away_eight_ball_match_points_earned": <pts>,
        // ... (various stats per rack)
        "winner": "home"
      },
      ...
    ],
    "location_id": "<loc_id>"
  },
  ...
}
```

This captures each team's total points (`team_scores`) and a detailed list of each game (`sets`), including which players faced off, their skill levels, points, and winner. These fields are built from the APA match scorecard API [9] [10].

- **Locations:** Under `"locations"`, each key is a **location ID** (string). Each entry has the bar's info:

```
"locations": {
  "<loc_id>": {
    "id": <loc_id>,
    "display_name": "Bar Name",
    "phone": "(123)456-7890",
    "address": { "street": "...", "city": "...", ... },
    "league_id": <league_id>
  },
  ...
}
```

This comes from querying APA location details [11].

In summary, the JSON is a hierarchy linking **divisions → teams → players** and also ties matches to teams and players. IDs connect across sections: each team has a `division_id`, each team's `roster` lists `player_id` slugs, each match points to `home_team_id` / `away_team_id` and inside each set uses the same player slugs. The division and team slugs themselves include format (_8 or _9) so that 8-ball vs 9-ball data stay distinct [4].

Data Relationships

- **Division ↔ Teams:** Each division entry lists which teams are in it (`team_ids`) and each team has a `division_id` linking back. Divisions aggregate standings of those teams.

- **Team ↔ Players:** Each team's `roster` lists players by slug and alias ID [6]. The same player slug appears under `"players"`, so the app can look up a player's name and stats. (Because players may have separate APA alias IDs per format, the code uses names to merge them into a single slug.)
- **Player Stats by Format:** A key point is that statistics are stored under each player split by format. For example, the code assigns entries under `"8-ball"` and `"9-ball"` in the player's `stats` object [7]. This matches APA's system where a person can have one alias for 8-ball and another for 9-ball. The structured data preserves this separation so you can query, say, *John Doe's* 8-ball record versus his 9-ball record.
- **Matches and Rosters:** Each match lists the `home_player_id` and `away_player_id` for every game. Those IDs match the `player_id` used in teams' rosters and in the `"players"` section. So from a match you can trace which rosters those players came from.

In practice, the Flask app would use these ID links to join data. For instance, to display a team's page, you'd look up the team by slug in `"teams"`, get its `roster` array, then fetch each player's info from `"players"`. Likewise, to show a division's standings, you'd list the teams in that division (by slug) using the data in `"divisions"` and `"teams"`. The naming convention (appending `_8` or `_9` to slugs) ensures teams and divisions for different formats don't collide [4].

Flask App Usage and Search

A Flask app can load this JSON into memory and expose pages and search endpoints. For example:

- **Load Data:** On startup, read `dump_structured_v2.json` into a Python dict. You might cache it and refresh weekly when new data is dumped.
- **Division Pages:** Create routes like `/divisions/<div_id>` to show a division's info (name, format, night) and standings. Use the division slug to index into `data["divisions"]`. List each team (`team_id`) and link to its page.
- **Team Pages:** At `/teams/<team_id>`, show the team's name, division, home location, and roster. Look up `data["teams"][team_id]` [5]. Render the roster table by fetching each player's name from `data["players"]` and their stats. Show the team's total points from the division standings (in `division["standings"]`).
- **Player Pages:** A route `/players/<player_id>` can display a player's profile. Index `data["players"][player_id]` to get name, profile, and stats. Present separate tables for 8-ball vs 9-ball stats (from the `stats["8-ball"]` and `stats["9-ball"]` objects). Also list the player's teams or matches if needed.
- **Matches/Schedule:** Endpoints like `/matches` (list all) or `/matches/<match_id>` to show a single scorecard. For each match, fetch `data["matches"][match_id]` [9] and display date, location, teams and scores. The detailed set info (`sets` array) can be shown as a series of game results. Link players back to their pages via the `home_player_id` / `away_player_id` fields.
- **Search:** Implement search forms to query by name or ID. For example, to find a player or team by name substring, scan through the `display_name` fields in `data["players"]` or `data["teams"]`. Return matches (similar to a Typeahead search). Because each entity has a unique `id` and human-friendly name, the app can map a search result to a URL (e.g. player slug to `/players/<slug>`).
- **Navigation:** Provide menus or links to browse by division, by format (8-ball vs 9-ball), or by location. The location list (`data["locations"]`) can link back to teams playing there.

Together, this lets a coach or league admin "browse" all data: view any team's roster and weekly points, any player's stats, or any match history. Weekly updates are handled by regenerating the JSON dump (via the transform script) and reloading it into the app.

APA Organization and Roles

The American Poolplayers Association (APA) is the **national league organization** for amateur pool in the U.S., with over 250,000 members [12]. It operates through a network of **franchise owners called League Operators (LOs)**. Each League Operator is authorized by APA to run local leagues in a geographic area [13]. LOs have full authority over schedules, standings, skill assignments and rules enforcement in their area. They may hire assistants or representatives, and collectively they form the *Local League Management*. Only an APA-authorized LO may administer a league [13].

Each team within a division has a **Team Captain** (the first name listed on roster) who manages the team and communications. The team captain's duties include collecting dues, submitting scoresheets, ensuring on-time attendance, and disseminating announcements [14]. Above the captains, the LO typically appoints **Division Representatives (DRs)** for each division. A DR is an active member who can answer rule questions and assist with local league matters, though they do not have formal ruling power [15]. The LO may also establish a **Board of Governors (BOG)** – a group (often including DRs and senior players) that advises on local policies, bylaws, protests, and sportsmanship issues [16]. Together, the LO, DRs, and BOG form the local organizational structure.

The APA's official publications (Team Manual and rules) define the standard policies. Local leagues may adopt **Local By-Laws** with APA's approval for area-specific rules. The Rio Grande Valley APA, for instance, publishes its local by-laws (approved by APA) supplementing the national rulebook [17].

Rio Grande Valley APA (Local League)

The Rio Grande Valley (RGV) APA is the local league for the McAllen/Harlingen area. Its website notes that *"RGV APA is the world's largest amateur pool league…"* and lists **Erin Lacy** as the League Operator [18]. (Erin's contact info is shown on the official site [19] and in the RGV By-Laws [20].) The RGV APA offers multiple formats each session – for example, "8-Ball Open", "8-Ball Doubles", "9-Ball Open" and "9-Ball Doubles". The seasons run in sessions (e.g. Spring, Summer, Fall 2025 shown on the site).

RGV's local by-laws (accessible via their website) outline any special rules for the league. They state that **"These Bylaws have been approved by the American Poolplayers Association"** and are a "secondary source of information" in addition to the official Team Manual [17]. The by-laws list the League Operator (Erin Lacy) and a contact phone number [20]. They would cover details like session dates, any roster change limits, fees, playoff formats, etc. (For instance, the by-laws mention that teams eligible for playoffs may have roster freeze periods to ensure fair play.)

In summary, the Flask app and JSON data reflect the APA structure: leagues are run by local LOs, each session is a season, players compete on teams in divisions (8-ball or 9-ball), and local roles (captains, DRs, BOG) coordinate league operations. By aligning the data model (divisions, teams, players) with APA's real organization, the app can serve as a coach's dashboard for the RGV APA.

Sources: We derived the JSON schema and relationships by inspecting the dump transformer script (e.g. how teams, players, matches are structured [5] [9]). Context on APA and RGV roles comes from official APA materials [13] [14] and the RGV APA website/bylaws [18] [17].

---

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] unload_dump_v2.py
file://file-MzDA9WsYcszQFSQG6n1Tja

[12] [18] [19] About
https://rgv.apaleagues.com/About.aspx

[13] League Operator - Pool Rules for APA League Play
https://rules.poolplayers.com/league-structure/league-operator/

[14] [15] [16] League Organization - Pool Rules for APA League Play
https://rules.poolplayers.com/league-structure/league-organization/

[17] [20] apa-by-laws.s3.amazonaws.com
https://apa-by-laws.s3.amazonaws.com/prod/785_bylaws_2025.pdf